

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 吴竞宸 学号 2024K8009929022 专业 计算机科学与技术
实验项目编号 4.1 实验名称 处理器性能增强设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- 设计步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- 开发步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- 调试步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

1 五级流水线处理器总体设计

本实验在 prj3 多周期处理器的基础上, 将指令执行过程切分为五级流水线: 取指(IF)、译码(ID)、执行(EX)、访存(MEM)、写回(WB), 并通过流水线寄存器在各级之间传递数据与控制信号。

流水线各级的核心功能如下: IF 级负责从指令 Cache 获取指令并更新 PC; ID 级完成指令译码、寄存器读取以及立即数生成; EX 级完成 ALU 运算、移位运算或乘法运算, 并在本级产生分支判断结果与跳转目标地址; MEM 级负责与数据 Cache 交互完成 load/store 操作; WB 级将结果写回寄存器文件。

为了提高流水线效率并保证程序正确性, 本设计完整实现了以下三项关键机制:

- **数据前递(Forwarding):** 当后续指令的源寄存器与前面指令在同一流水线寄存器中的目的寄存器匹配时, 直接将 ALU 结果或写回数据前递给 EX 级, 避免等待写回。
- **Load-Use 阻塞(Stall):** 当 load 指令的目的寄存器被紧随其后的指令作为源操作数使用时, 插入一个周期的流水线气泡, 等待 load 数据从 MEM 级返回。
- **跳转冲刷(Flush):** 在 EX 级判定分支是否跳转, 若跳转则冲刷 IF/ID 和 ID/EX 流水线寄存器, 并从目标地址重新取指。

流水线控制采用经典的 valid-ready-allowin 握手协议。每一级以 valid 信号指示本级是否有有效数据, 以上一级的 allowin 信号指示是否可以接收新数据。流水线前进条件为 `valid && readygo && allowin_next`, 其中 readygo 在 ID 级受 load-use 检测控制, 在 MEM 级受访存握手等待控制。

2 数据前递与 Load-Use 阻塞设计

数据前递是本设计中最关键的优化。在 RISC-V 五级流水线中,寄存器在 ID 级读取但结果在 WB 级才写回。当相邻指令存在 RAW(读后写)数据依赖时,后续指令将读到旧的寄存器值。本设计采用从前级流水线寄存器直接前递数据的方式解决此问题。

前递的来源有两个:EX/MEM 流水线寄存器中的 ALU 结果,以及 MEM/WB 流水线寄存器中的写回数据。前递的目的地为 EX 级的两个源操作数 rdata1 和 rdata2。检测逻辑在 EX 级展开,比较 ID_EX 流水线寄存器中的读地址(raddr1、raddr2)与 EX/MEM 和 MEM/WB 中的目标寄存器编号(rd),若匹配且前级确实要写寄存器,则选择对应的前递数据。

前递的优先级规则为:EX/MEM 前递优先于 MEM/WB 前递。这是因为在 RAW 依赖链上,更近的前级结果才是程序实际需要的值。这一优先级的实现方式是在 MEM/WB 前递的触发条件中附加“未被 EX/MEM 前递覆盖”的条件。

Load-Use 冒险是一种前递无法解决的特殊情况。Load 指令的数据在 MEM 级结束时才从内存获得,而依赖该数据的使用指令在 EX 级就需要操作数,中间差了一个时钟周期。本设计通过在 ID 级检测这种依赖关系,若 EX 级或 MEM 级流水线寄存器中有未完成的 load 指令且其目标寄存器被当前 ID 级指令作为源操作数引用,则向 IF/ID 流水线寄存器插入一个气泡(NOP),使流水线暂停一个周期。

3 分支与跳转处理

本设计在 EX 级完成分支判断和跳转目标地址计算。一旦判定分支跳转(Branch taken)、JAL 或 JALR,EX 级产生 jump_flush 信号,该信号触发三件事:第一,将 IF/ID 流水线寄存器内容清零(插入 NOP),并将 ID_valid 置为 0;第二,清空 ID/EX 流水线寄存器中的控制信号(如 MemRead、MemWrite、RegWrite、分支和跳转标志位);第三,PC 被更新为目标地址。

在取指端,由于指令 Cache 可能尚未准备好响应(Inst_Req_Ready 延迟),还引入了 discard 机制:若跳转发生时有一条新指令正在从 Cache 读取(I_W 状态),但指令数据尚未到达,则需要将接下来到来的那条指令标记为 discard 并丢弃,替换为 NOP,避免错误指令进入流水线。

跳转目标地址的计算方式因指令类型而异:JAL 和条件分支使用 PC+ 立即数偏移(在 ID/EX 流水线寄存器保存当前指令的 PC 和立即数),JALR 使用 ALU 计算结果的低 31 位(最低位清零以保证地址对齐)。

4 乘法与移位指令支持

本设计在 EX 级集成了乘法器和移位器,通过操作码和 funct3/funct7 字段译码选择正确的执行单元。乘法指令(MUL)通过 ID_is_mul 信号识别,结果由组合逻辑乘法(src1 * src2)产生;移位指令(SLL、SRL、SRA)通过 ID_is_shift_imm 信号识别,由 shifter 模块执行。ALUop 和 Shiftop 的控制信号在 ID 级根据 opcode 和 funct3/funct7 并行译码产生,与指令数据一起通过 ID/EX 流水线寄存器传递至 EX 级。

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 流水线控制与握手信号

流水线的核心控制逻辑包括 valid 信号管理、readygo 判定和 allowin 链:

```
wire MEM_waiting = ((MemRead || Read_data_Ready) && !Read_data_Valid)
                || (MemWrite && !Mem_Req_Ready);
wire load_in_EX = ID_EX_MemRead && EX_valid && (ID_EX_rd != 5'b0)
                && ((ID_EX_rd == raddr1) || (ID_EX_rd == raddr2));
```

```

wire load_in_MEM = EX_MEM_MemRead && MEM_valid && (EX_MEM_rd != 5'b0)
                    && ((EX_MEM_rd == raddr1) || (EX_MEM_rd == raddr2));
wire load_use_stall = ID_valid && (load_in_EX || load_in_MEM);

wire IF_readygo = Inst_Valid && !discard;
wire ID_readygo = !load_use_stall;
wire EX_readygo = 1'b1;
wire MEM_readygo = !MEM_waiting;
wire WB_readygo = 1'b1;

wire WB_allowin = 1'b1;
wire MEM_allowin = !MEM_valid || MEM_readygo && WB_allowin;
wire EX_allowin = !EX_valid || EX_readygo && MEM_allowin;
wire ID_allowin = !ID_valid || ID_readygo && EX_allowin;

wire IF_ID_advance = IF_readygo && ID_allowin;
wire ID_EX_advance = ID_valid && ID_readygo && EX_allowin;
wire EX_MEM_advance = EX_valid && EX_readygo && MEM_allowin;
wire MEM_WB_advance = MEM_valid && MEM_readygo && WB_allowin;

```

其中 MEM_waiting 信号体现了流水线处理器与真实内存的交互。在 MEM 级,若进行读操作时 Read_data_Valid 尚未有效,或进行写操作时 Mem_Req_Ready 尚未就绪,则 MEM_readygo 为低,流水线在 EX/MEM 处阻塞。这种阻塞会沿 allowin 链反向传播,最终导致整条流水线暂停。

readygo 的判断在 ID 级包含了 load-use 检测。当解码出当前指令的源寄存器与 EX 级或 MEM 级中未完成的 load 指令的目标寄存器匹配时,ID_readygo 为低,阻止当前指令进入 ID/EX 流水线寄存器。同时,IF_readygo 中包含了 discard 信号的检查,确保在跳转冲刷期间错误取出的指令不会进入 ID 级。

2 数据前递与旁路逻辑

前递检测的逻辑位于 EX 级,在进入 ALU 之前选择正确的操作数来源:

```

wire forward_a_EXMEM = EX_MEM_RegWrite && MEM_valid && !EX_MEM_MemRead
                    && (EX_MEM_rd != 5'b0) && (EX_MEM_rd == ID_EX_raddr1);
wire forward_a_MEMWB = MEM_WB_RegWrite && WB_valid
                    && (MEM_WB_rd != 5'b0) && (MEM_WB_rd == ID_EX_raddr1)
                    && !forward_a_EXMEM;
wire forward_b_EXMEM = EX_MEM_RegWrite && MEM_valid && !EX_MEM_MemRead
                    && (EX_MEM_rd != 5'b0) && (EX_MEM_rd == ID_EX_raddr2);
wire forward_b_MEMWB = MEM_WB_RegWrite && WB_valid
                    && (MEM_WB_rd != 5'b0) && (MEM_WB_rd == ID_EX_raddr2)
                    && !forward_b_EXMEM;

```

这里特别注意前递触发条件的完备性:只有当目标寄存器编号不为零(x0 寄存器恒为零,不应该前递),且 MEM 级确实有效且将要写寄存器时,前递才会生效。对 MEM/WB 前递的附加条件!forward_a_EXMEM 和!forward_b_EXMEM 保证了 EX/MEM 前递具有更高的优先级。

MEM/WB 前递的数据不是直接使用 MEM_WB 流水线中的某个字段,而是通过一个综合了写回选择信号(WB_sel)的多路选择器产生 fwd_data,确保无论 ALU 结果、load 数据、返回地址还是立即数,都能正确地作为前递数据送给 EX 级。

另外,当流水线因 load-use 阻塞而暂停时,ID/EX 流水线寄存器的内容保持不变,但 rdata1 和 rdata2 可能

因为前级写回而改变。因此在 ID_EX_advance 为假但 EX_valid 为真(流水线暂停)时,需要根据最新的前递条件更新 ID_EX_rdata1 和 ID_EX_rdata2,确保在流水线恢复后 EX 级使用的是最新的寄存器值。

3 分支判断与 PC 更新

```
wire Branch = ID_EX_is_branch && (
    ((ID_EX_funct3 == 3'b000) && Zero)    | // BEQ
    ((ID_EX_funct3 == 3'b001) && !Zero)   | // BNE
    ((ID_EX_funct3 == 3'b100) && ALU_Result[0]) | // BLT
    ((ID_EX_funct3 == 3'b101) && !ALU_Result[0]) | // BGE
    ((ID_EX_funct3 == 3'b110) && ALU_Result[0]) | // BLTU
    ((ID_EX_funct3 == 3'b111) && !ALU_Result[0]) // BGEU
);
wire EX_taken = EX_valid && (Branch || ID_EX_is_jal || ID_EX_is_jalr);

wire [31:0] next_pc =
    ({32{ID_EX_is_jal}}      & (ID_EX_PC + ID_EX_imm)) |
    ({32{ID_EX_is_jalr}}    & {ALU_Result[31:1], 1'b0}) |
    ({32{ID_EX_is_branch & Branch}} & (ID_EX_PC + ID_EX_imm)) |
    ({32{!((ID_EX_is_branch && Branch) || ID_EX_is_jal
        || ID_EX_is_jalr)}} & (pc_reg + 32'd4));
```

PC 更新使用了条件多路选择的位拼接方式,所有非跳转路径最终通过默认条件选择 pc_reg+4。这种写法避免了嵌套的 if-else 逻辑,综合结果更加清晰。

跳转冲刷时,ID/EX 流水线寄存器中的控制信号(MemRead、MemWrite、RegWrite、分支和跳转标志位)被选择性清零,防止这些信号继续向后传播触发错误的访存或写回操作。

4 性能计数器

```
reg [31:0] cycle_cnt;
assign cpu_perf_cnt_0 = cycle_cnt;
always @ (posedge clk) begin
    if (rst) cycle_cnt <= 32'd0;
    else cycle_cnt <= cycle_cnt + 32'd1;
end
```

性能计数器从复位释放后每个时钟周期自增,记录程序执行的总周期数。同时,inst_retire 信号在每条指令完成 WB 级时记录该指令的 PC、目标寄存器和写回数据,供上层性能分析使用。

三、 学习总结(总结设计、开发与调试过程中遇到的问题, 及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	逻辑综合	握手信号	大量超时	握手信号置位错误	修正
2	逻辑电路设计	前递/旁路	数据依赖场景下计算结果错误	旁路逻辑条件不完备	修复
3	逻辑电路设计	分支/跳转	跳转后错误指令进入流水线	跳转冲刷逻辑错误	修复

表 1: 实验问题总结

● **握手信号置位错误补充。**rst 复位时要求 Inst_Ready、Read_data_Ready 拉高: 清空 CPU 访存通路, 避免复位释放后访存出错。但一直错误地置为 0, 导致仿真过上板测试只有第一项测试能过。

● **旁路逻辑错误补充。**除了 rd==0 的排除条件外, 另一个容易出错的地方是 load 指令的前递排除。Load 指令的结果在 MEM 级才获得, 前递到 EX 级不可行, 因此在 EX/MEM 前递条件中必须加入 !EX_MEM_MemRead。若遗漏此条件, load 指令会在 EX 级将错误的数据前递给后续指令。此外, 在流水线因 load-use 而暂停时, ID/EX 流水线中的 rdata 需要根据最新的前递条件动态更新, 否则解阻塞后 EX 级使用的仍是旧的寄存器值。

● **跳转冲刷错误补充。**跳转冲刷的时序需精确控制。discard 机制引入了额外的状态管理复杂度: 当 jump_flush 与取指请求同时发生时, 如果正在进行取指握手且指令数据尚未到达, 需要将 discard 置 1; 当指令数据到达时检查 discard 标志, 若为 1 则丢弃该指令并清零 discard。漏掉任何一条路径都会导致错误指令进入 ID 级。此外, jump_flush 也需要清除 ID/EX 流水线寄存器中的控制信号, 否则这些信号会继续向后流动, 触发错误的读写操作。

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

类别	名称	体会	后续计划
亮点	流水线握手协议设计	通过统一的协议管理五级流水线	继续使用
亮点	前递与阻塞协同	前递解决大多数数据依赖	考虑引入分支预测减少冲刷开销
不足	流水线调试方法	波形信号量大、依赖关系复杂, 定位 bug 慢	学习使用断言辅助验证

表 2: 收获总结

在课后, 你花费了大约 40 小时完成此次实验, 其中各个阶段的用时如表 3 所示。

步骤	时长(小时)	说明
设计	1	流水线架构设计与数据通路规划
开发	3	流水线控制逻辑实现
调试	36	前递逻辑与跳转冲刷的时序调试

表 3: 实验时长